

Cómputo Concurrente 2026

Práctica 6

Monitores y Consenso

Profa: Gilde Valeria Rodríguez

1 Contexto

El curso se ha desarrollado en un modelo de memoria compartida, un claro ejemplo son las computadoras multicore, la concurrencia también existe en otro tipo de modelos, como el modelo de paso de mensajes, un ejemplo son los sistemas de blockchain.

En ambos modelos se han enfocado en resolver el consenso, de hecho, hay protocolos de consenso muy famosos (Bitcoin, Ethereum, Algorand, etc). Sin embargo, solo en el modelo de memoria compartida wait-free tenemos una jerarquía del objeto menos poderoso al más poderoso, en base al número de consenso.

También, hemos considerado que los hilos son asíncronos, es decir, que un hilo se puede detener y reanudar en cualquier momento sin una garantía estricta de tiempo.

En general existen sistemas síncronos, asíncronos y semisíncronos (o parcialmente síncronos) [Lyn96]. En un sistema síncrono, existen límites conocidos para el tiempo de comunicación entre nodos y para la ejecución de cada proceso, lo cual garantiza un comportamiento predecible y facilita la coordinación. En contraste, un sistema asíncrono no tiene garantías de tiempo, por lo que la comunicación y la ejecución pueden experimentar retrasos indefinidos, esto implica que los algoritmos deben diseñarse para tolerar comportamientos impredecibles. Un ejemplo de un sistema asíncrono es Bitcoin [SARM24].

En un sistema parcialmente síncrono se establecen límites de tiempo, sin embargo, mientras no se cumplan el sistema funciona de forma asíncrona, un ejemplo de ello es Algorand [BLL⁺23] y Cardano [DGKR18].

En esta práctica vamos a simular un sistema que resuelve el consenso por rondas, de forma parcialmente síncrona (Programa en 3).

La idea es que existen n rondas, en cada ronda se crea una instancia de un objeto de consenso *protocolCAS* (programa en 1), los hilos ejecutan *protocolCAS.decide(id)* en cada ronda, una vez que todos los hilos hayan terminado inicia una nueva ronda.

Al final se tiene a la lista de ganadores en cada ronda.

Es parcialmente síncrono porque los hilos tratan de ganar el consenso de forma asíncrona, pero esperan a que todos hayan terminado para iniciar una nueva ronda, así que hay un límite de tiempo.

Para simular las rondas utilizaremos un monitor, el objeto *CountDownLatch* en Java (Programa en 2) permite coordinar la ejecución de hilos al hacer que uno o varios hilos esperen hasta que una serie de instrucciones hayan terminado. Funciona con un contador inicial que se reduce cada vez que se llama al método *countDown()*. Cuando el contador llega a cero, todos los hilos que estaban esperando en *await()* pueden continuar.

2 Ejemplos

En el siguiente link: https://github.com/surindt/FC_CConcurrente/tree/main/Programas_P6/unam.fc.concurrent.practica6/src/unam/fc/concurrent/practica6

1. Programa 1 (CASConsensus): Protocolo de consenso utilizando *compareAndSet()* y *get()*
2. Programa 2 (CountDownLatch): Monitor CountDownLatch, cuenta con dos métodos, *await()* espera a que el contador sea 0, *countDown()* disminuye el contador en 1
3. Programa 3 (ExecConsenRounds): Sistema de *Semisíncrono* en el cual en cada ronda se resuelve el consenso, utiliza *CASConsensus* y *CountDownLatch*
4. Programa 4 (FifoReadWriteLock): Monitor de los lectores/escritores justo, visto en clase
5. Programa 5 (ExecReadersWriters): Executor del monitor FifoReadWriteLock, utiliza dos runnables, uno para los lectores, otro para los escritores

3 Ejercicios

- Entrega en un pdf las respuestas de los ejercicios que no requieran implementarse y añade una breve descripción de los programas que entregas (sus nombres y qué hacen).
- Debes realizar un programa en cada ejercicio que indique “Implementar”.
- El formato y el medio de entrega los indicará tu ayudante de laboratorio.
- Recuerda que debes utilizar Java 21 LTS.
- **Si no compila utilizando Java 21LTS o si no se entrega la breve descripción de los programas se penalizará.**

Tiempo de elaboración: $\approx$ 2hr

Total de puntos: 100

1. [20 min / 15 puntos] Modifica el `compareAndSet()` por un `testAndSet()` en el programa *CASConsensus* (1). Argumenta si sigue resolviendo el consenso para 4 hilos, agrega una captura de pantalla de la ejecución que lo sustente. Justifica tu respuesta en no más de dos renglones.
2. [45 min / 45 puntos] La implementación en *ExecConsenRounds* (3) permite que solo unos hilos ganen el consenso. Implementa una *técnica de ayuda* similar al de la *Construcción universal wait-free* para que todos los hilos ganen el consenso eventualmente en alguna ronda. *Hint: Unos hilos ayudan a otros, puedes proponer un arreglo announce como en la construcción universal wait-free, y ayudar por % número de hilos.*
3. [30 min / 30 puntos] Considera el monitor *FifoReadWriteLock*, actualmente cumple con:
 - (a) Si un lector está en su sección crítica, ningún escritor puede entrar a su propia sección crítica
 - (b) Si un escritor está en su sección crítica, **ningún escritor o lector** puede entrar a su propia sección críticaImplementa el monitor modificando la propiedad (b) de forma que ahora se cumpla:
 - (a) Si un lector está en su sección crítica, ningún escritor puede entrar a su propia sección crítica
 - (b) Si un escritor está en su sección crítica, **ningún lector** puede entrar a su propia sección críticaNo es necesario que el protocolo sea justo.
4. [10 min / 10 puntos] ¿Cómo proporcionarías justicia a tu implementación del inciso anterior? No es necesario que lo implementes.

References

- [BLL⁺23] Erica Blum, Derek Leung, Julian Loss, Jonathan Katz, and Tal Rabin. Analyzing the Real-World Security of the Algorand Blockchain. 11 2023.
- [DGKR18] Bernardo David, Peter Gazi, Aggelos Kiayias, and Alexander Russell. Ouroboros praos: An adaptively-secure, semi-synchronous proof-of-stake blockchain. In Jesper Buus Nielsen and Vincent Rijmen, editors, *Advances in Cryptology – EUROCRYPT 2018*, pages 66–98, Cham, 2018. Springer International Publishing.
- [Lyn96] Nancy A. Lynch. *Distributed Algorithms*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 1996.
- [SARM24] Muhammad Saad, Afsah Anwar, Srivatsan Ravi, and David Mohaisen. Revisiting nakamoto consensus in asynchronous networks: A comprehensive analysis of bitcoin safety and chain quality. *IEEE/ACM Transactions on Networking*, 32(1):844–858, 2024.